

Unit IV - Linked List

Introduction to Static and Dynamic Memory Allocation

- **Memory allocation** is the process of assigning specific areas in a system's memory to a programming language or code written for execution.
- The term allocation is used to indicate variable has been stored.
- Memory is divided into two parts known as **stack Memory and heap Memory**.
- **Stack Memory** is associated with **static Memory allocation** and is smaller than heap Memory. It is also known as **compile time allocation**.
- **Heap Memory** is associated with **dynamic Memory allocation** and is larger than stack Memory. It is also known as **run-time allocation**.

Static Memory Allocation

- Static Memory allocation in C/C++ allocates size and location to a fixed variable.
- Static Memory allocation is done before the program is executed.
- In other words, memory is allocated during compile time.
- Hence, it is known as compile time allocation.
- The size and the location of the variable are fixed throughout the program.
- Example of Static Memory Allocation in C++
- Let us look at an example of allocating static Memory in C++

```
#include <iostream>
using namespace std;
int main()
{
    int p;
    char c;
    int arr[100];
    return 0;
}
```

Dynamic Memory Allocation

- In dynamic Memory allocation in C++, the Memory is defined during the execution of the program.
- Heap Memory is associated with dynamic Memory allocation and is larger than stack Memory. It is also known as run-time allocation.
- **Pointers** play a crucial role in dynamic Memory allocation in C++.
- In dynamic Memory allocation, the programmer must do the memory allocation and de-allocation manually.
- The '**new**' keyword is used to allocate Memory, and the '**delete**' keyword is used to deallocate Memory.
- While creating a new variable, if Memory is available, then Memory is initialized, and the address of the space is returned to the pointer variable.
- Different factors need to be considered-
 1. Memory reuse is possible in dynamic Memory allocation.
 2. Memory leak is possible. It needs to be taken care of by the programmer.

Allocate memory in C++:

- **Syntax-**

```
pointer_variable = new data_type;
```

The pointer is of pointer data_type. The data_type is of the type float, int string, etc.

Example -

```
int *k = new int;  
int *k = new int(19);
```

De-allocate memory in C++:

- **Syntax-**

```
delete pointer_variable;
```

Example-

```
delete k;
```

Example of Dynamic Memory Allocation in C++

```
#include <iostream>

using namespace std;

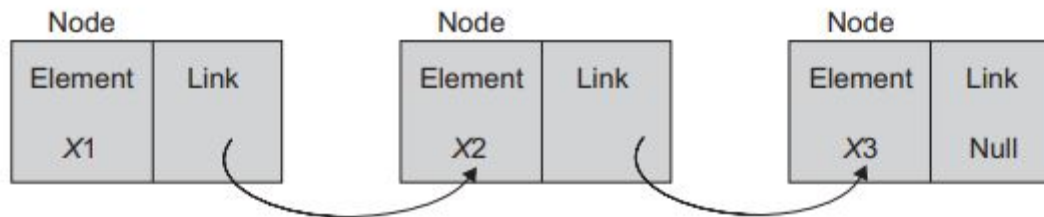
int main()
{
    int n = 19;
    // Allocate memory.
    int * arr = new int[n];
    cout << sizeof(arr) << endl;
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    // Free up space.
    delete[] arr;
    return 0;
```

Differences between Static and Dynamic Memory Allocation

Static memory allocation	Dynamic memory allocation
Static memory allocation in C++ allocates size and location to a fixed variable.	Dynamic memory allocation in C++ allocates size and location to a dynamic variable.
Static memory allocation is done before the program is executed.	In dynamic memory allocation in C++, the memory is defined during the execution of the program.
It is known as compile time allocation.	It is known as run-time allocation.
It is less flexible.	It is more flexible.
Stack memory is used.	Heap memory is used.
Allocation and de-allocation are done by the compiler.	Allocation and de-allocation are done by the programmer using new and delete keywords.

Linked List: Introduction

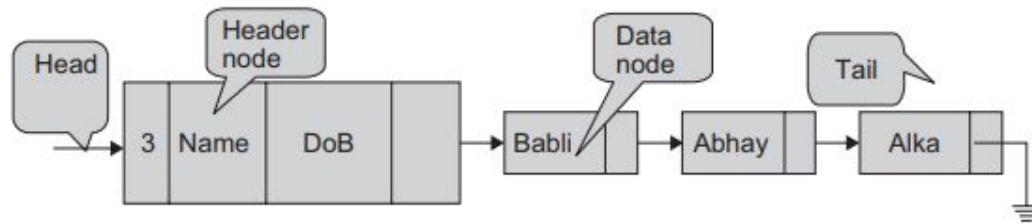
- The linked list is a linear data structure that contains a sequence of elements such that each element links to its next element in the sequence. Each element in a linked list is called "**Node**".
- A linked list is an ordered collection of data in which each element (node) contains a minimum of two values, data and link(s) to its successor (and/or predecessor).
- A list with one link field using which every element is associated to its successor is known as a singly linked list (SLL).
- In a linked list, before adding any element to the list, a memory space for that node must be allocated. A link is made from each item to the next item in the list.



Linked List Terminology

The following terms are commonly used in discussions about linked lists:

- **Header node** -A header node is a special node that is attached at the beginning of the linked list. This header node may contain special information (metadata) about the linked list .



- **Data node** -The list contains data nodes that store the data members and link(s) to its predecessor (and/or successor).
- **Head pointer** -The variable (or handle), which represents the list, is simply a pointer to the node at the head of the list. A linked list must always have at least one pointer pointing to the first node (head) of the list. This pointer is necessary because it is the only way to access the further links in the list. This pointer is often called head pointer, because a linked list may contain a dummy node attached at the start position called the header node.
- **Tail pointer** Similar to the head pointer that points to the first node of a linked list, we may have a pointer pointing to the last node of a linked list called the tail pointer.

Realization of linked list using dynamic memory management

- Linked lists need not be stored in adjacent locations.
- Individual elements can be stored anywhere in the memory.
- Each data element is called a node.
- Each node contains at least two fields namely data and link. Every node holds a link to the next node in the list.
- During run-time, as per the need, a node is allocated.
- In other words, a new node of the list will be created dynamically.
- We just remember the pointer to the list at the end, that is, pointer to the first node.
- In addition, the last node's link field can be set to 0 to mark the end of the list. The 0 here represents null.
- A linked list thus maintains the data elements in a logical order rather than in a physical order or in other words separates the physical view from the logical view.

Primitive Operations

The following are basic operations associated with the linked list as a data structure:

1. Creating an empty list
2. Inserting a node
3. Deleting a node
4. Traversing the list

Some more operations, which are based on the basic operations, are as follows:

5. Searching a node
6. Updating a node
7. Printing the node or list
8. Counting the length of the list
9. Reversing the list
10. Sorting the list using pointer manipulation
11. Concatenating two lists
12. Merging two sorted lists into a third sorted list

Linked List as ADT

- A linked list, when implemented as an Abstract Data Type (ADT), is a data structure that encapsulates the linked list operations and provides a high-level interface for users.
- The key idea behind an ADT is to hide the implementation details and expose only the necessary methods or functions for interacting with the data structure.
- In this ADT for a linked list, you have the following essential methods and features:
 1. **Create and Delete:** The constructor initializes the linked list, and the destructor is responsible for cleaning up memory to prevent memory leaks.
 2. **Add:** Adds a new element at the end of the list.
 3. **Prepend:** Inserts a new element at the beginning of the list.
 4. **Insert:** Inserts a new element at a specific position in the list.
 5. **Remove:** Removes the first occurrence of a specified value from the list.
 6. **Get:** Retrieves the value at a specific position in the list.
 7. **Size:** Returns the number of elements in the list.
 8. **IsEmpty:** Checks if the list is empty.
 9. **Display:** Displays the elements in the list.

Types of Linked List

Linked lists can be classified broadly as follows:

- 1. Singly linked list / Linear linked list**
- 2. Circular Linked List**
- 3. Doubly linked list**
- 4. Doubly Circular Linked List**

1. Singly Linked List

- Singly Linked List A linked list in which every node has one link field, to provide information about where the next node of the list is, is called as singly linked list (SLL).
- It has no knowledge about where the previous node lies in the memory.
- In SLL, we can traverse only in one direction.
- We have no way to go to the i th node from $(i + 1)$ th node, unless the list is traversed again from the first node.



2.Circular Linked List

- Although a linear linked list is a useful and popular data structure, it has some shortcomings. For example, consider an SLL. Given a pointer A to a node in a linear list, we cannot reach any of the nodes that precede the node to which A is pointing.
- This disadvantage can be overcome by making a small change.
- This change is without any additional data structure.
- The link field of the last node is set to **Null** in a linear list to mark the end of the list.
- This link field of the last node can be set to point to the first node rather than **Null**.

Such a linked list is called a circular linked list



- From any node in such a list, it is possible to reach to any other node in the list.
- We need not traverse the list again right from the first node.
- Circular linked list is used in many applications.
- Circular linked list is used to keep track of free space (unused nodes) in memory.
- In a circular list, traversal can be continued from current node.
- It helps us to keep the traversal procedure an unending one.
- The two primary applications of circular list is time slicing and memory management.

3. Doubly Linked List

- In a doubly linked list (DLL), each node has two link fields to store information about the one to the next and also about the one ahead of the node.
- Hence, each node has knowledge of its successor and also its predecessor.
- In DLL, from every node, the list can be traversed in both the directions.
- A DLL may either be linear or circular and it may or may not contain a header node.
- DLLs are also called two-way lists.

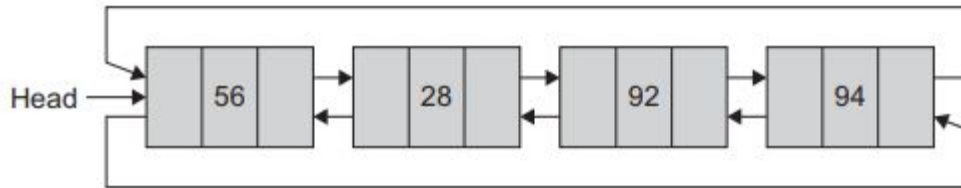


Node structure of doubly linked list



4. Doubly Circular Linked List

- In doubly circular linked list, the last node's next link is set to the first node of the list and the first node's previous link is set to the last node of the list.
- This gives access to the last node directly from the first node.



Singly Linked List and operations on SLL

Data structure of node

- Each node has data and link fields.
- The data field holds data element(s) and the link field(s) stores the address of its successor (and/or predecessor, if any).
- As the link field is a pointer to its successor, it should be a pointer variable, which should hold the address of its successor. The successor node is of the same type as that of the node itself.
- The declaration of the data structure of a node is given as follows:

```
class Node
{
    public:
        int data;
        Node *link;
};

class List
{
    private:
        Node *Head;
    public:
        .
        . } member functions here
        .
};
```

- Here, within the class, the statement `Node *link` defines the link field of a node. Here,
- `Node` is a data type of the pointer variable `link`.

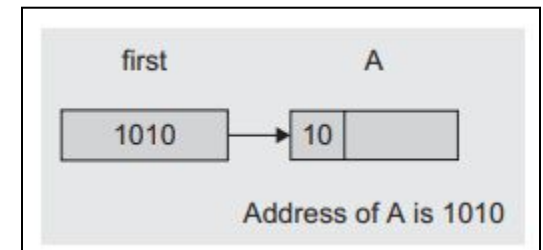
Consider the following piece of code:

```
class Node
{
    public:
        int data;
        Node *link;
} *first, A;
first = &A;
A.data = 10;
A.link = Null;
```

Now, the statement

```
cout << first->data;
```

will print the output 10.



//create a singly linked list

//use keyword struct

//The structure of the 1st node will be:

//the first data member/ will be an integer

*// the second member of the node will be a node pointer
called next*

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
#include <iostream>
using std::cin;
using std::cout;
```

```
// Structure of the node of a singly linked list
```

```
typedef struct node
{
```

```
    int data;
    node* next;
```

```
}node;
```

```
// Function prototypes
```

```
node *Create(int value);
```

```
node *InsertAtBeg(node *head, int value);
```

```
node *InsertAtPos(node *head, int value, int position);
```

```
node *InsertAtEnd(node *head, int value);
```

```
node *DeleteAtBeg(node *head);
```

```
node *DeleteAtPos(node *head, int position);
```

```
node *DeleteAtEnd(node *head);
```

```
node *Concatenate(node *head1, node *head2);
```

```
int Search(node *head, int element);
```

```
void Traverse(node *head);
```

```
int main()
{
    node *head1 = new node;
    // Creating a head node of value 81
    head1 = Create(81);
    // Inserting a node of value 2 at the end
    head1 = InsertAtEnd(head1, 2);
    // Inserting a node of value 53 at the beginning
    head1 = InsertAtBeg(head1, 53);
    // Inserting a node of value 44 at the 2nd position
    head1 = InsertAtPos(head1, 44, 2);
    // Printing the first linked list
    cout<<"FIRST: ";
    Traverse(head1);
    return 0;
}
```

Creation of Linked List

*// Function to create the head node (first node) of a singly
Linked List*

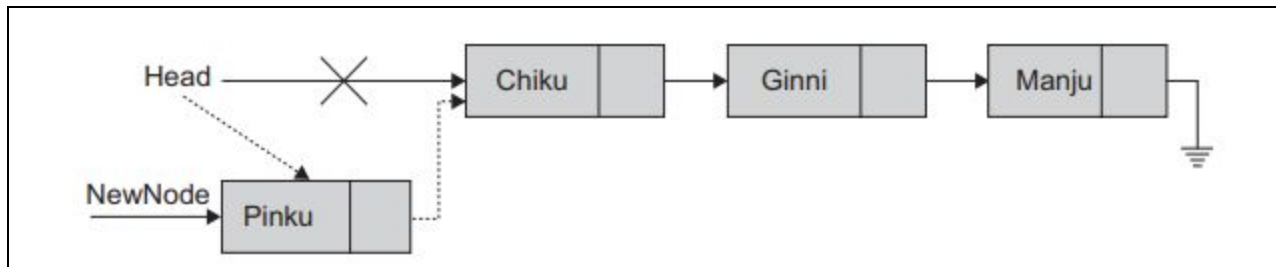
```
node *Create(int value)
{
    node *ptr = new node;
    ptr->data = value;
    ptr->next = NULL;
    return ptr;
}
```


Insertion of a node in SLL

- Depending on the type of list or need of the user, insertion can be made at the **beginning, middle, or at the end of the list.**
- If the list is an ordered list, the insertion should not affect the order and this may require inserting the data at proper locations so that the order is preserved.
- The information about where the node is to be inserted can be decided by searching through the list, obtaining the position, and then inserting the same.

Insertion of a Node at the First Position

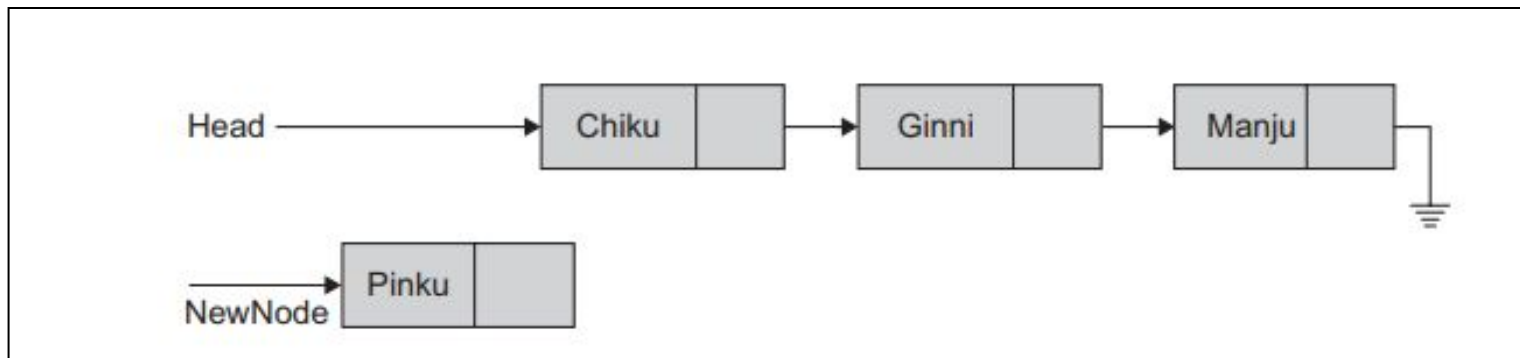
- Let us consider a situation when the node is to be inserted at the first position.
- Insertion of a node at the middle, we need Prev, which is a pointer to the node after which NewNode is to be added.
- To insert a node at the first position, there exists no Prev node.
- The link manipulations needed to add a node at the first location is shown in Fig using dotted lines.



- Head is the pointer variable pointing to the starting node of the list.
- The insertion of NewNode at the first position should make Head point to NewNode, and in addition, the current node which is at the first position should become the second node of the list. Hence, the link field of NewNode should be set to point to the current first node, that is, the node pointed by the pointer First.
- The following two steps will insert NewNode at the beginning of the linked list.

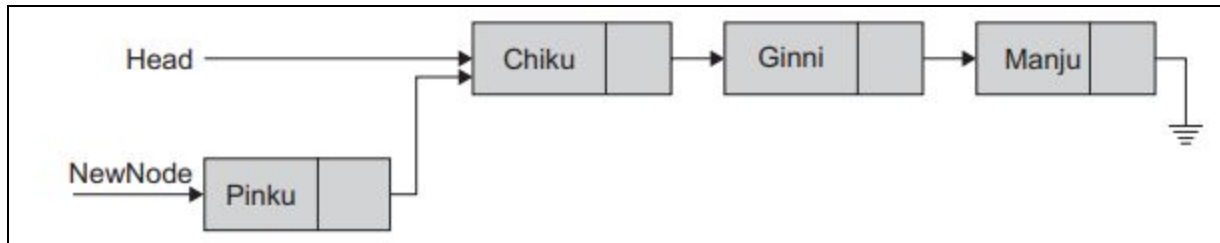
NewNode->link = Head;

Head = NewNode;



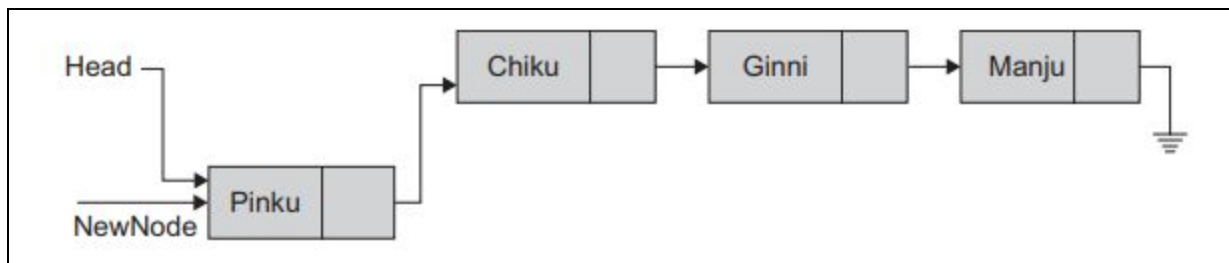
Step 1 This step is represented in Fig.

`NewNode->link = Head;`



Step 2 This step is represented in Fig .

`Head = NewNode;`



// Function to insert a new node at the beginning of a singly

Linked List

```
node *InsertAtBeg(node *head, int value)
```

```
{
```

```
    node *ptr = new node;
```

```
    ptr->data = value;
```

```
    ptr->next = head;
```

```
    head = ptr;
```

```
    return head;
```

```
}
```

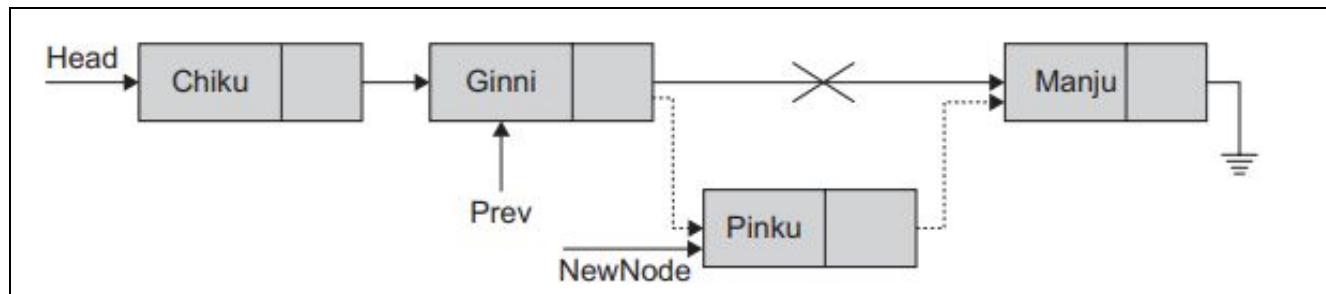
Insertion of a Node at a Middle Position

- Assume that a node is to be inserted at some position other than the first position. Let Prev refer to the node after which NewNode node is to be inserted.
- We need the following two steps:

`NewNode->link = Prev->link;`

`Prev->link = NewNode;`

- The node NewNode is to be inserted between Prev and the successor of Prev.
- The link manipulation required to accomplish this is shown in Figure with dotted lines.



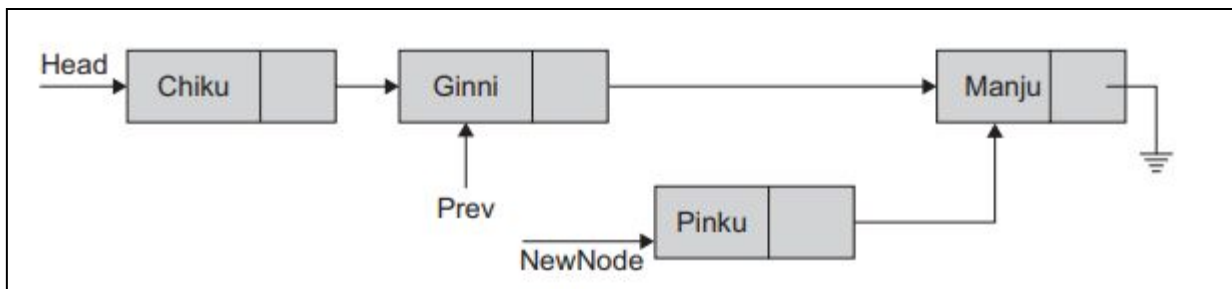
The steps to perform the link manipulation are as follows:

Step 1:

- NewNode is a node to be inserted after Prev.
- The node that is a successor of Prev will now become the successor of NewNode.
- Currently, Prev->link holds the pointer to the successor of Prev.
- Set the link field of the NewNode such that Prev's successor node becomes the successor of NewNode.

NewNode->link = Prev->link;

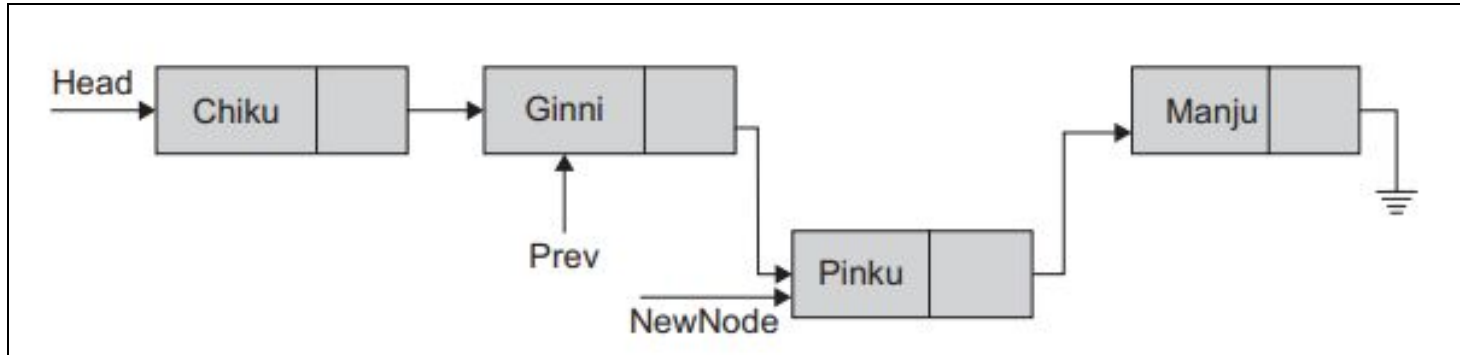
- In other words, NewNode becomes the predecessor of the node whose predecessor was Prev, because NewNode is to be placed in between Prev and its successor



Step 2.

- Now, let us make NewNode the successor of Prev. This can be achieved by setting the link field of Prev to NewNode.

`Prev->link = NewNode;`



// Function to insert a new node at the specified position of a singly linked list

node *InsertAtPos(node *head, int value, int position)

```
{
    if(position < 1)
    {
        cout<<"Invalid Position";
        return head;
    }

    if(position == 1)
        return InsertAtBeg(head, value);

    node *ptr1 = new node;
    ptr1 = head;

    node *ptr2 = new node;
    ptr2->data = value;
    ptr2->next = NULL;

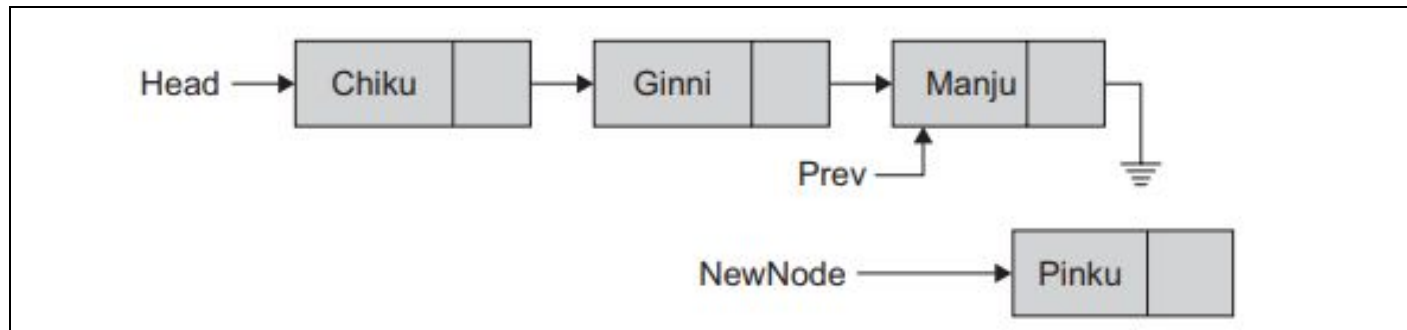
    while(--position > 1)
        ptr1 = ptr1->next;

    ptr2->next = ptr1->next;
    ptr1->next = ptr2;

    return head;
}
```

Insertion of a Node at the End

- The steps for inserting a node in the middle of a list also work for inserting a node at the end of the list.
- As the node is to be inserted after the last node, Prev is a pointer to the last node.
- Let the node to be inserted be NewNode as shown in Fig.



Step 1.

`NewNode->link = Prev->link`

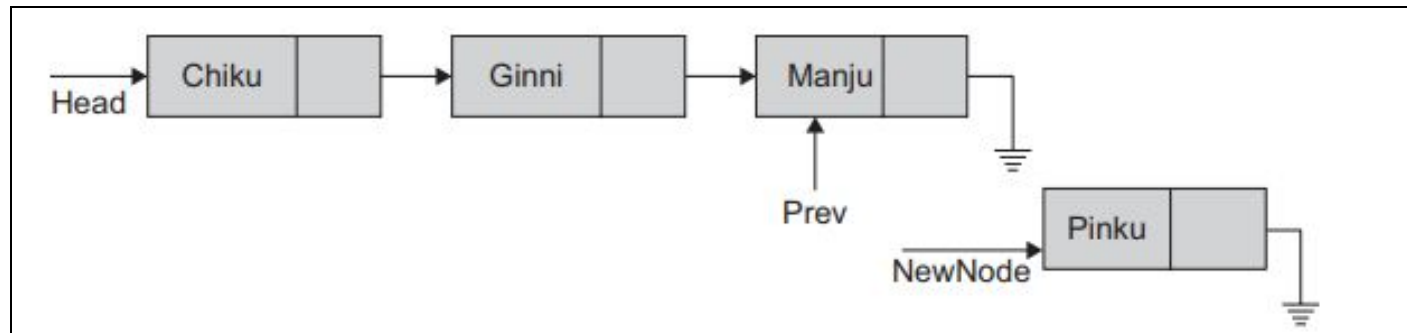
- As Prev is the last node,

`Prev->link = Null`.

- Hence, this step can be replaced by the statement

`NewNode->link = Null`

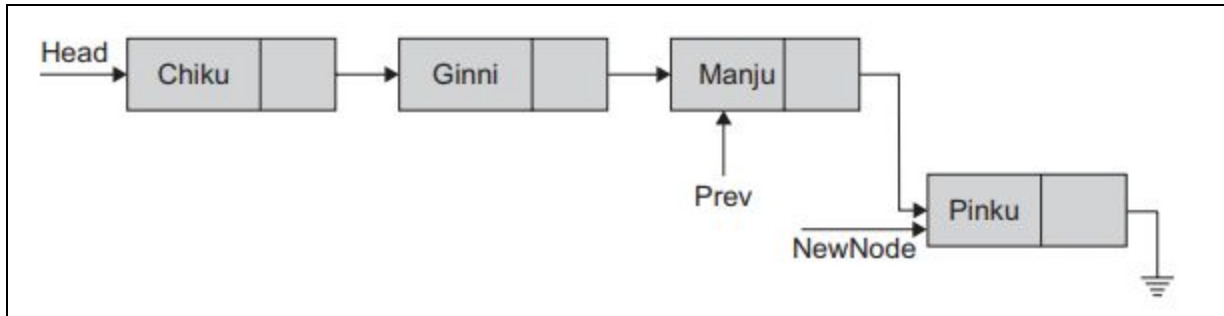
- if we know that Prev is the last node as in Fig.



Step 2.

`Prev->link = NewNode;`

- This will make the node NewNode the successor of Prev. This is shown in Fig.



- This will insert the node NewNode at the last position, that is, make the node NewNode the last node of the list.

*// Function to insert a new node at the end of a singly
Linked List*

```
node *InsertAtEnd(node *head, int value)
{
    node *ptr1 = new node;
    ptr1 = head;

    while(ptr1->next != NULL)
    {
        ptr1 = ptr1->next;
    }
    node *ptr2 = new node;
    ptr2->data = value;
    ptr2->next = NULL;

    ptr1->next = ptr2;

    return head;
}
```

Linked List traversal

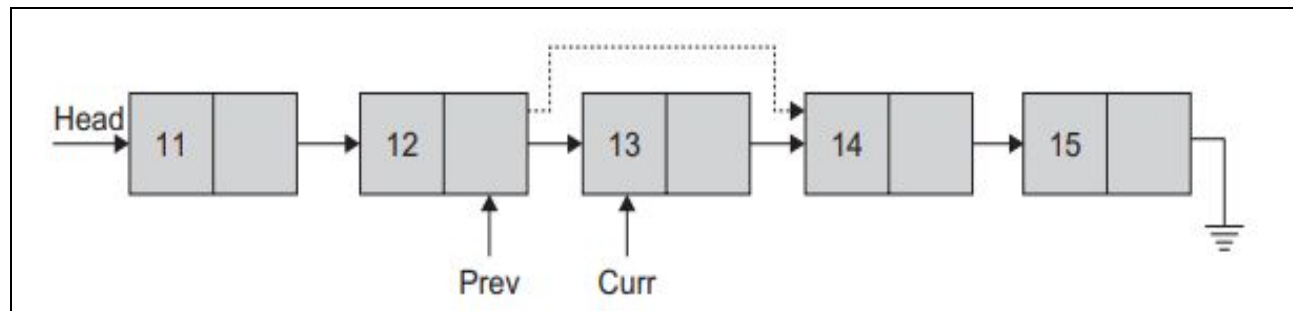
- List traversal is the basic operation where all elements in the list are processed sequentially, one by one. Processing could involve retrieving, searching, sorting, computing the length, and so on.
- **Algorithm**
 1. Get the address of the first node, call it current;
`current = Head`
 2. if current is **Null**, goto step 6.
 3. Process the data field of the current node (node pointed by current). Here, the process may include printing data, updating, and so on.
 4. Move to the next node –
`current = current->link`
goto step 2
 5. stop

// Function to print a singly linked list from head node to tail node

```
void Traverse(node *head)
{
    node *ptr = head;
    cout << "The linked list: ";
    while (ptr != NULL)
    {
        cout << ptr->data << " ";
        ptr = ptr->next;
    }
}
```

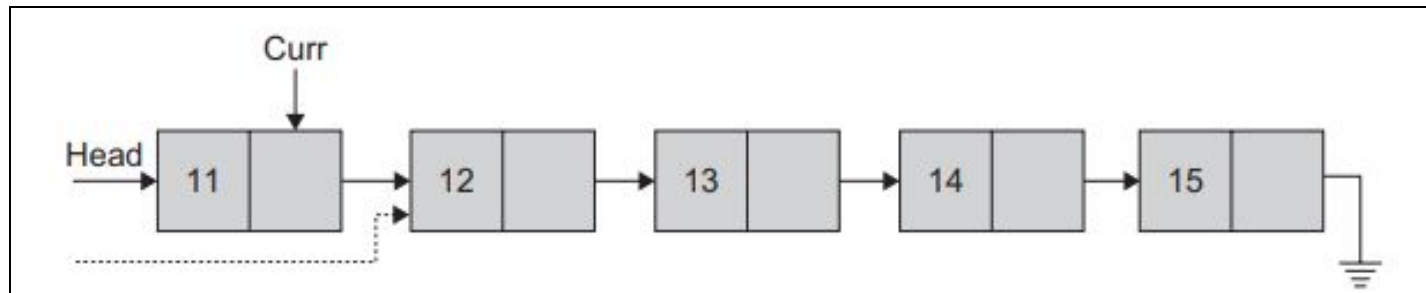
Deletion of a Node

- There may be nodes that are to be deleted from a list.
- Let us assume that the node to be deleted contains data x.
- We need the following steps to delete the same.
- Let $x = 13$ and let it be pointed to by the pointer Curr.
- To delete this node, the required link manipulations are shown in Fig. with dotted lines.
- To delete the node Curr, we need to modify the link between Curr and its previous node, and the link between Curr and its successor.



Deleting the First Node

- Deleting the first node is also referred to as deleting a header node.
- If the node at the first position is to be deleted, then we need to modify the pointer pointing to the first node say Head.
- Deletion of the first node needs the link manipulations shown in Fig. with dotted lines.



Step 1.

- Set another pointer to the first node before modifying Head, which is the pointer pointing to the first node.

- Set Head to point to the second node. This can be accomplished by the statements,

```
Curr = Head;
```

```
Head = Head->link;
```

Step 2.

- Now, release the memory allocated for the first node.

```
delete Curr;
```

- These two statements will delete the first node, and Head will point to the second node so that the second node becomes the first node.

// Function to delete a node from the beginning of a singly Linked List

```
node *DeleteAtBeg(node *head)
{
    if(head == NULL)
    {
        cout<<"Underflow! Can't delete from empty list";
        return head;
    }
    node* ptr = new node;
    ptr = head;
    head = head->next;
    delete ptr;
    return head;
}
```

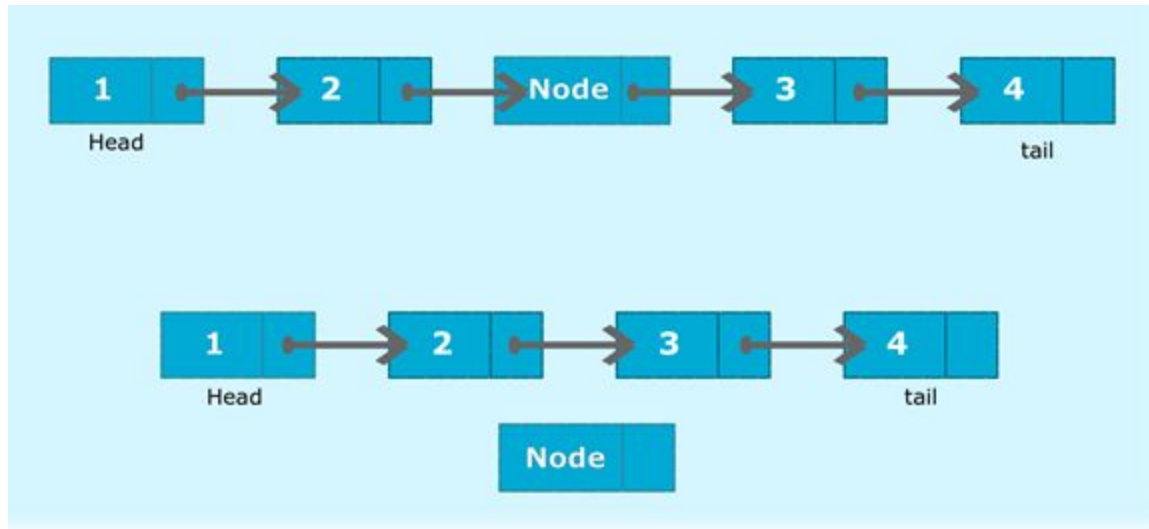
Deleting a Middle Node

- Let curr point to the node to be deleted, and prev be the predecessor of curr. Then, the following statements will delete the node curr.

```
prev->link = curr->link;
```

```
delete curr;
```

- These two statements will also delete the last node of the list.



// Function to delete a node from the specified position of a singly linked list

```
node *DeleteAtPos(node *head, int position)
{
    if(head == NULL)
    {
        cout << "Underflow! Can't delete from empty list";
        return head;
    }

    if(position < 1)
    {
        cout << "Invalid Position";
        return head;
    }
```

```
if(position == 1)
    return DeleteAtBeg(head);

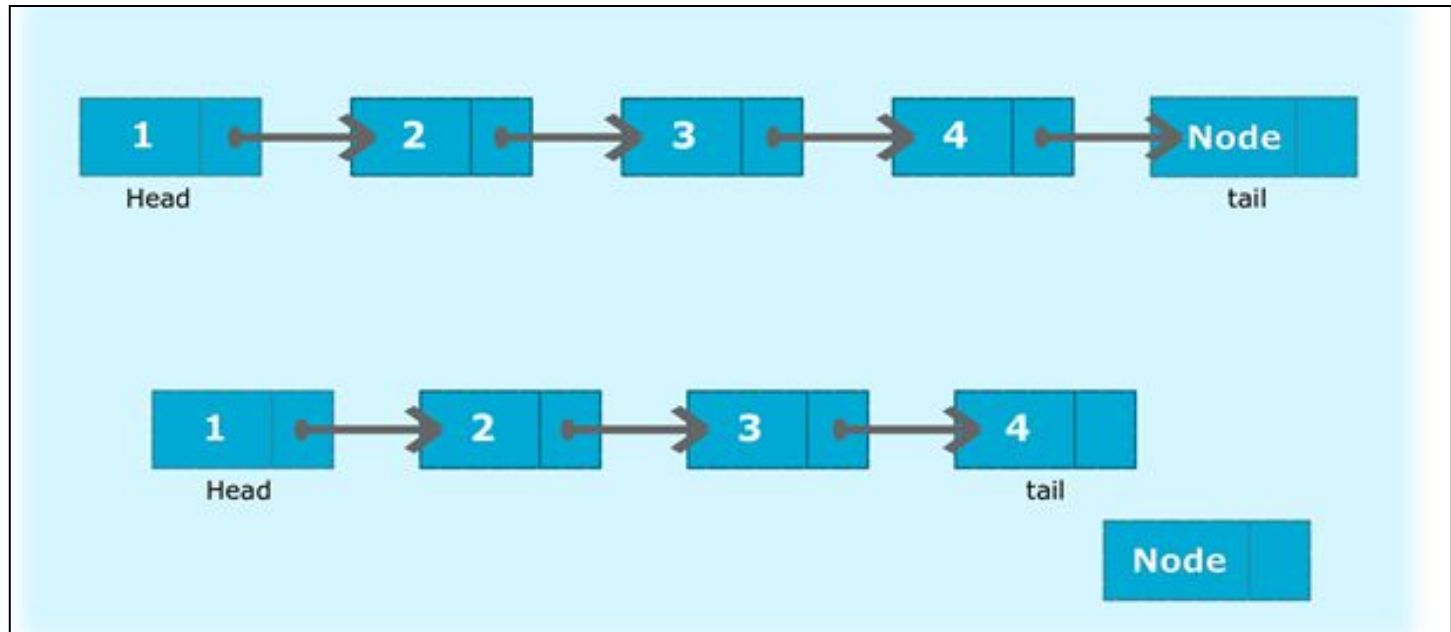
    node *ptr1 = new node;
    ptr1 = head;

    position--;
    while(position > 1)
    {
        ptr1 = ptr1->next;
        position--;
    }

    node *ptr2 = new node;
    ptr2 = ptr1->next;
    ptr1->next = ptr2->next;
    delete ptr2;
    return head;
}
```

Deleting the Last Node

- To accomplish this task, we first find out the second last node of the list. Then, make second last node as the new tail of the list. Then, delete the last node of the list.



Algorithm

1. It first checks whether the head is null (empty list) then, display the message "List is empty" and return.
2. If the list is not empty, it will check whether the list has only one node.
3. If the list has only one node, it will set both head and tail to null.
4. If the list has more than one node then, traverse through the list till node current points to second last node in the list.
5. Node current will become the new tail of the list.
6. Node next to current will be made null to delete the last node.

*// Function to delete a node from the end
of a singly linked list*

```
node *DeleteAtEnd(node *head)
{
    if(head == NULL)
    {
        cout << "Underflow! Can't delete
from empty list";
        return head;
    }
    if(head->next == NULL)
    {
        delete head;
        return NULL;
    }

    node *ptr1 = new node;
    ptr1 = head;
```

```
while(ptr1->next->next != NULL)

    ptr1 = ptr1->next;

    node *ptr2 = new node;

    ptr2 = ptr1->next;

    delete ptr2;

    ptr1->next = NULL;

    return head;

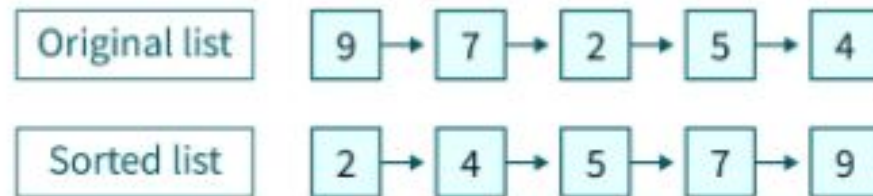
}
```


// Function to search for an element in a singly linked list

```
int Search(node *head, int element)
{
    node *ptr = new node;
    ptr = head;
    int position = 0;
    while (ptr != NULL)
    {
        position++;
        if (ptr->data == element)
            return position;
        ptr = ptr->next;
    }
    return 0;
}
```

Sorting Linked List

- For sorting a linked list, we can use an insertion sort kind of technique. We can use two points (let's say current and nextNode are those two pointers). At first, the current pointer will be to the head of the linked list and the nextNode pointer will point to the second node of the linked list (or the next node to the current node).
- Now, we would traverse the entire linked list (until the current node is not equal to null), and in each step, we will check if the current node's data is greater than a node's data. In case it is larger, we will swap the data between them.
- In each iteration, we will select one current node and place the currently selected node at its correct sorted position in the given linked list.



Algorithm to Sort Linked List

We need to call the **sortLinkedList()** function. Inside the **sortLinkedList()** function, we need to perform certain steps:

1. We will first create two nodes namely **current** and **nextNode** .
2. The **current** node will be pointing at the **head** of the linked list.
3. The **nextNode** will be pointing to the second node of the linked list.
4. In the next step, we need to compare all the nodes of the linked list with the **current** node. If the data of the **current** node is greater, then we need to swap the data of the current node and the currently processing node.
5. After the swapping, the current node will point to the next node to the current node (i.e. `current->next`).
6. We will continue the entire above steps until the linked list is sorted.

// Function to sort a singly linked list using insertion sort

```
void insertionSort(Node* headref)
```

```
{
```

// Initialize sorted linked list

```
sorted = NULL;
```

```
Node* current = headref;
```

// Traverse the given linked list and insert every node to sorted

```
while (current != NULL)
```

```
{
```

// Store next for next iteration

```
Node* next = current->next;
```

// Insert current in sorted linked list

```
sortedInsert(current);
```

// Update current

```
current = next;
```

```
}
```

// Update head_ref to point to sorted linked list

```
head = sorted;
```

```
}
```

/ Function to insert a new_node in a list. Note that this function expects a pointer to head_ref as this can modify the head of the input linked list (similar to push()) */*

```
void sortedInsert(Node* newnode)
```

```
{
```

```
    // Special case for the head end
```

```
    if (sorted == NULL ||
```

```
        sorted->val >= newnode->val)
```

```
    {
```

```
        newnode->next = sorted;
```

```
        sorted = newnode;
```

```
    }
```

```
    else
```

```
    {
```

```
        Node* current = sorted;
```

```
        /* Locate the node before the point of insertion */
```

```
        while (current->next != NULL &&
```

```
            current->next->val < newnode->val)
```

```
        {
```

```
            current = current->next;
```

```
        }
```

```
        newnode->next = current->next;
```

```
        current->next = newnode;
```

```
    }
```

```
}
```

// Function to concatenate two singly linked lists

```
node *Concatenate(node *head1, node *head2)
{
    node *ptr = new node;
    ptr = head1;

    while (ptr->next != NULL)
    {
        ptr = ptr->next;
    }
    ptr->next = head2;
    return head1;
}
```

Creation of doubly Linked List

The structure representing a node of the doubly linked list is defined as:

// structure to define a node of the doubly linked list in C++

```
typedef struct node  
{
```

// part which will store data

```
int data;
```

// pointer to the previous node

```
struct node *prev;
```

// pointer to the next node

```
struct node *next;
```

```
}node;
```

// Create a new node

```
node* createNode(int data) {  
    node* newNode = new node;  
    if (newNode == NULL) {  
        cout<<"Failed to allocate memory for a new node";  
        exit(1);  
    }  
    newNode->data = data;  
    newNode->prev = NULL;  
    newNode->next = NULL;  
    return newNode;  
}
```


Insertion in Doubly Linked List in C++

- Insertion in a doubly linked list in C++ means we create a new node with some data and add it to the linked list. Now, there can be three ways of inserting a node in a doubly linked list in C++. These are:

- 1. Insertion of node at the front of the list**
- 2. Insertion of node after a given node of the list**
- 3. Insertion of node at the end of the list**

At the Front of the Doubly Linked List

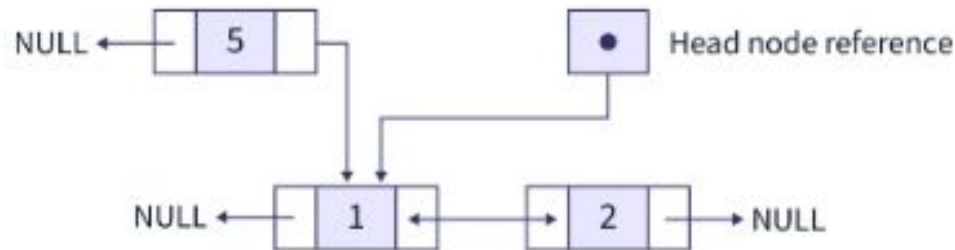
- The steps to add a new node at the front of a doubly linked list are:
 1. Create a new node with the required data value.
 2. Set the previous pointer of this new node to NULL, as it will be added to the front of the doubly linked list and will not have any other node in front of it.
 3. Set the next pointer of this new node to the current head of the doubly linked list as it will be placed in front of the head node.
 4. The previous pointer of the current head node should be set to point to this new node.
 5. The new node has been successfully added to the doubly linked list at the beginning. The diagram below illustrates the complete method.



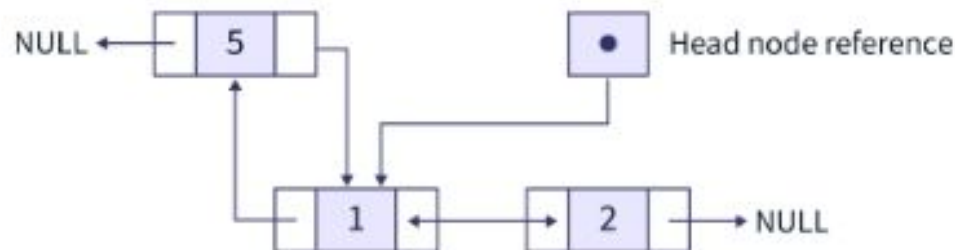
1. New node



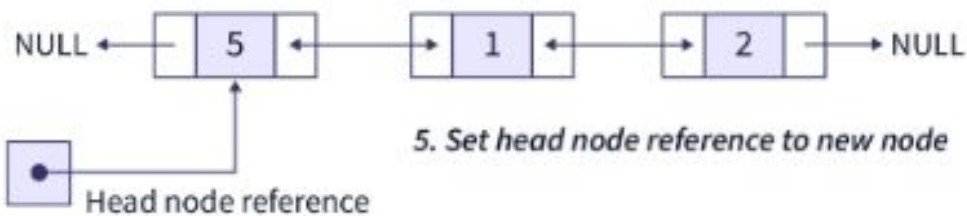
2. Set previous to NULL



3. Set next to point to head node



4. Set previous of current head to new node



5. Set head node reference to new node

// Insert a node at the beginning of the list

```
node* insertAtBeginning(node* head, int data)
```

```
{
```

```
    node* newNode = createNode(data);
```

```
    newNode->next = head;
```

```
    if (head != NULL)
```

```
    {
```

```
        head->prev = newNode;
```

```
    }
```

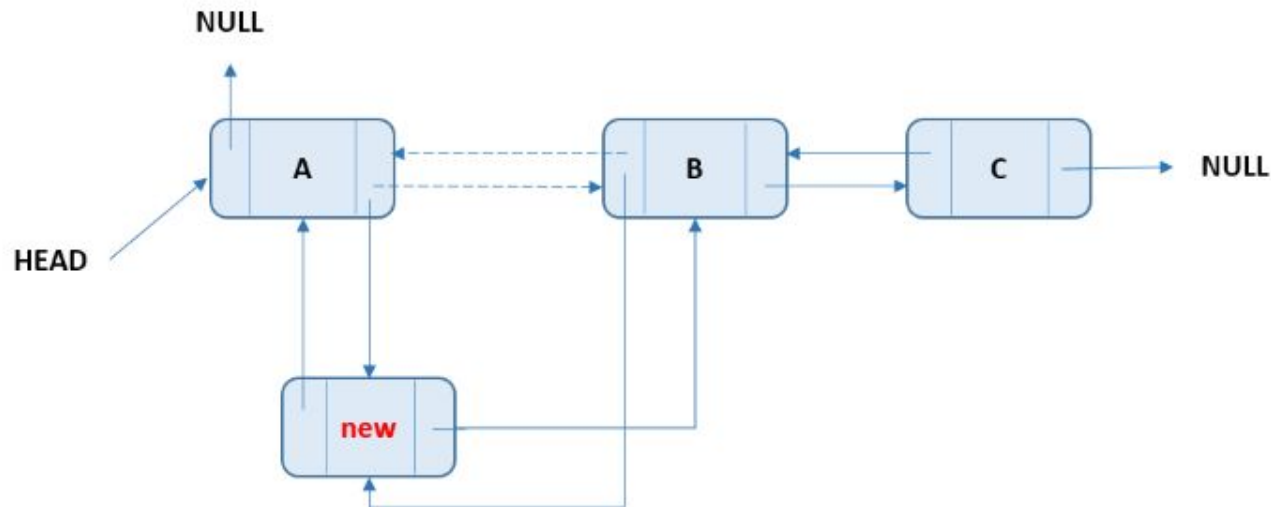
```
    head = newNode;
```

```
    return head;
```

```
}
```

Insert a new node at a given position in the Doubly Linked List

1. First, a new node with given element is created.
2. If the insert position is 1, then the new node is made to head.
3. Otherwise, traverse to the node that is previous to the insert position and check if it is null or not.
4. In case of null, the specified position does not exist.
5. In other case, update the links.
6. The below figure describes the process, if the insert node is other than the head node.



// Function to insert a new node at the specified position of a DLL

node *InsertAtPos(node *head, int value, int position)

```
{
    if(position < 1)
    {
        cout<<"Invalid Position";
        return head;
    }

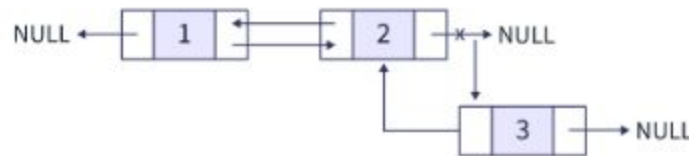
    if(position == 1)
        return insertAtBeginning(head, value);

    node *ptr1 = new node;
    ptr1 = head;
    node *ptr2 = new node;
    ptr2->data = value;
    ptr2->next = NULL;
    ptr2->prev=NULL;
    while(--position > 1)
        ptr1 = ptr1->next;

    ptr2->next = ptr1->next;
    ptr2->prev=ptr1;
```

At the End of the DLL

- We will be given the starting or the head of the doubly linked list as input. In such a case, the steps to add a new node at the very end of a doubly linked list are:
 1. Create a new node with the required data value.
 2. As we are adding the new node at the end of the doubly linked list, there will be no nodes after it so set its next pointer to NULL.
 3. If the doubly linked list is already empty, then this new node will become the head. So, set its previous pointer to NULL and assign head pointer to this node.
 4. Else, we travel to the end of the linked list, so our head pointer points to the last node in the linked list.
 5. As our new node will be after this last node, set the next pointer of this last node to the new node, and the previous pointer of the new node to this last node.



// Insert a node at the end of the list

```
node* insertAtEnd(node* head, int data)
```

```
{
```

```
    node* newNode = createNode(data);
```

```
    if (head == NULL)
```

```
{
```

```
        head = newNode;
```

```
}
```

```
else
```

```
{
```

```
    node* current = head;
```

```
    while (current->next != NULL)
```

```
{
```

```
        current = current->next;
```

```
}
```

```
    current->next = newNode;
```

```
    newNode->prev = current;
```

```
}
```

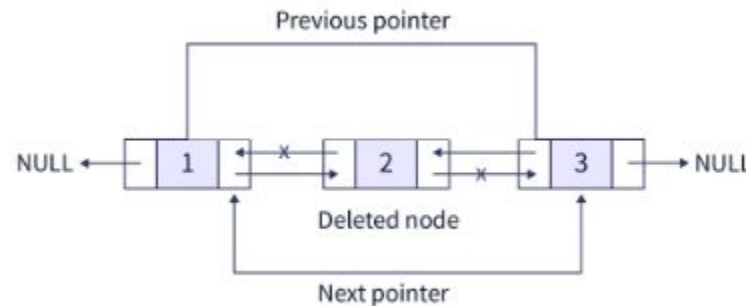
```
return head;
```


Delete a Node in DLL

In a singly linked list, we need the pointer to the node before the node we want to delete. But in a doubly linked list we have the previous pointer in the node itself.

Let us look at the steps to delete a node of a doubly linked list in C++.

1. Check if this node is the head node. If yes, then simply assign head to the next of this node.
2. Else, assign the next pointer of the previous node to point to this nodes next.
3. If the next pointer of the given node points to NULL, it means the node is the last node, and it is deleted.
4. Else, make the previous pointer of the next node point to the previous node of the given node.

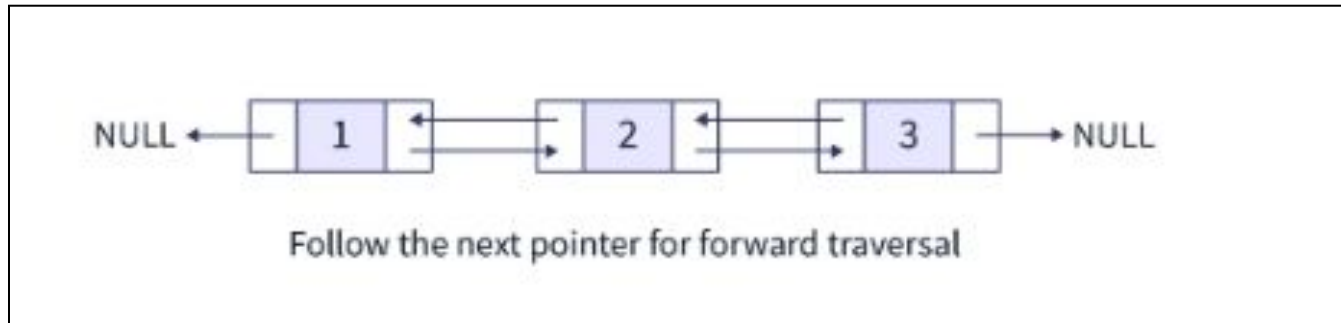


// Delete a node with a given data value

```
node* deleteNode(node* head, int data) {
    if (head == NULL) {
        printf("List is empty, nothing to delete.\n");
        return head;
    }
    node* current = head;
    node* t1,*t2;
    while (current != NULL) {
        if (current->data == data) {
            if (current->prev != NULL) {
                t1= current->prev;
                t1->next = current->next;
            }
            else
            {
                head = current->next;
            }
            if (current->next != NULL) {
                t2 =current->next;
                t2->prev = current->prev;
            }
            delete current;
            return head;
        }
        current = current->next;
    }
}
```

Traversing a DLL

- In a singly linked list we know we can travel only in the forward direction, but as a doubly linked list has both previous and next pointers, we can travel in both forward and reverse directions in a doubly linked list in C++.
- Let us look at both of these traversals one by one.
- **Forward Traversal**
 - In forward traversal, we just need to follow the next pointers of the nodes of the doubly linked list until we reach the end, which is denoted by the NULL pointer. This can be easily achieved using a while loop in C++.



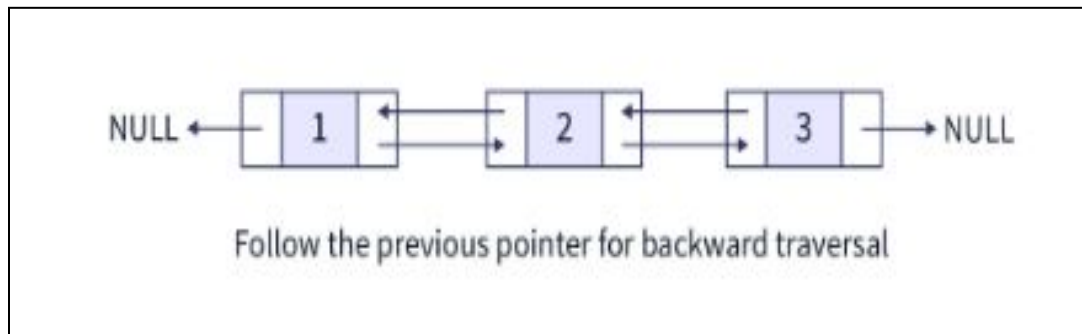
```
void Backward_traversal(node* head)
{
    // print the list in backward direction
    cout<<"In backward direction:"<<endl;

    while(head !=NULL)
    {
        end = head;
        head = head->next;
    }

    while(end != NULL)
    {
        cout<<end->data<<" ";
        end = end->prev;
    }
    return 0;
}
```

Backward Traversal

- Backward traversal is same as forward traversal, the only difference is we need to use the previous pointers of the nodes of a doubly linked list until we reach the start of the doubly linked list, which is again denoted by the NULL pointer. This can also be achieved with a simple while loop in C++.



```
void Forward_traversal(node* head)
{
    // print the list in forward direction
    cout<<"In forward direction:"<<endl;

    while(head != NULL)
    {
        end = head;
        cout<<head->data<<" ";
        head = head->next;
    }

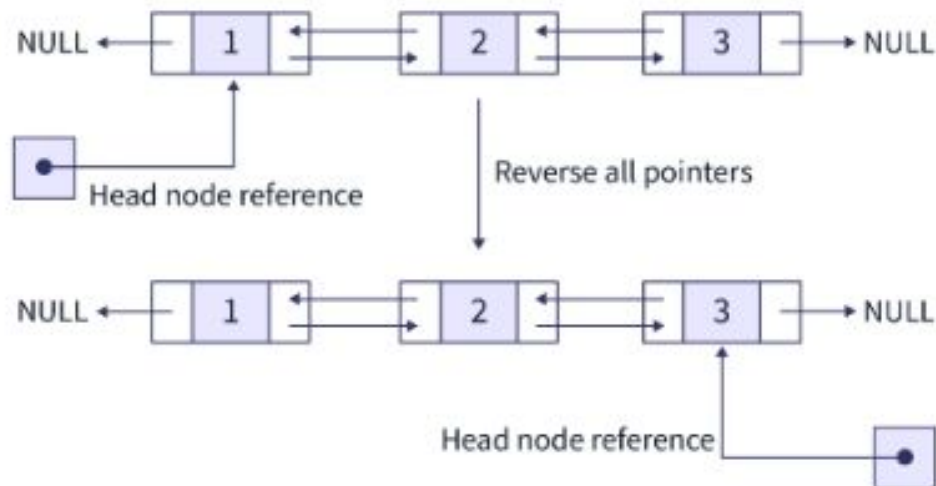
    return 0;
}
```

Searching a Node in DLL

```
// function to search given target
void search(node** head, int target){
    // node to traverse the Linked List
    node* temp = (*head);
    // variable to store index
    int index = 1;
    // loop over the List
    while(temp != NULL)
    {
        // if target is found
        if(temp->data == target)
        {
            cout<<"Found the target at node "<<index<<endl;
            return;
        }
        // go to next node
        index++;
        temp = temp->next;
    }
    // node wasn't found
    cout<<"Target not found"<<endl;
}
```

Reverse a DLL

Reversing a doubly linked list involves reversing or swapping the next and previous pointers of all the nodes. So basically to reverse a doubly linked list we need to traverse the linked list and just swap the next and previous pointers of all the nodes.



// function to reverse the doubly linked list

```
void reverse(node* head)
```

```
{
```

```
    node* temp = NULL;
```

```
    node* current = head;
```

// to reverse the list we just swap the next and prev of all the nodes

```
    while (current != NULL) {
```

```
        temp = current->prev;
```

```
        current->prev = current->next;
```

```
        current->next = temp;
```

```
        current = current->prev;
```

```
    }
```

// edge case

```
    if (temp != NULL)
```

```
        head = temp->prev;
```

```
}
```

Advantages of DLL over the Singly Linked List

- A doubly linked list has both next as well as previous pointer unlike singly linked list which has only the next pointer.
- The doubly linked list can be traversed in both the forward as well in the backward direction as opposed to singly linked list which we can travel only in the forward direction.
- We can delete a node in a doubly linked list by just having the pointer to the node to be deleted. In a singly linked list we need the node before the node to be deleted.

Disadvantages of DLL over the Singly Linked List

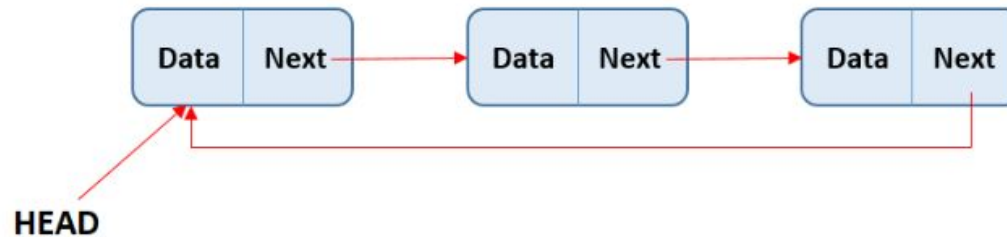
- The operations on a doubly linked list, such as insertion and deletion are complex because of the presence of two pointers, next and previous in every node.
- Each node of the doubly linked list has to store two pointers, previous and next which takes up more memory space as compared to a singly linked list.

Applications of Doubly Linked List

- A doubly linked list has the most useful application of being used in systems where backward and forward traversals are required.
- Web browsers use a doubly linked list data structure to implement going back to the website you last visited, or to go forward in your search history as well.
- Undo and Redo functionality can also be implemented by storing all operations performed in the form of a doubly linked list data structure.
- Many thread schedulers also use a doubly linked list to schedule processes.
- Other data structures such as hash maps, binary trees and stacks can also be implemented using a doubly linked list.

Circular Linked List in C++

- The structure of the circular linked list is exactly similar to the structure of the singly linked list. It will contain two variables first one is of primitive or user-defined data type to store the data at the current node and another is a pointer to store the reference or address of the next node.



// creating a structure using the struct keyword

```
Typedef struct Node
```

```
{
```

// declaring a variable to store the data

```
int data; // it could be of any data type (declaring integer here)
```

// declaring a pointer to store the address of the next node

```
struct Node* next; // here struct is the keyword to indicate the
```

pointer of Node type

```
}Node;
```

Operations On Circular Linked List:

There are various operations that can be performed on a circular linked list. Some of the operations are listed below.

1. Insertion
 1. At beginning
 2. At the end
 3. In between
2. Deletion
 1. From beginning
 2. From the end
 3. In between
3. Traversal
4. Searching

Insertion in a circular linked list:

A node can be added in three ways:

1. Insertion in an empty list
2. Insertion at the beginning of the list
3. Insertion at the end of the list
4. Insertion in between the nodes

Insertion in a circular linked list:

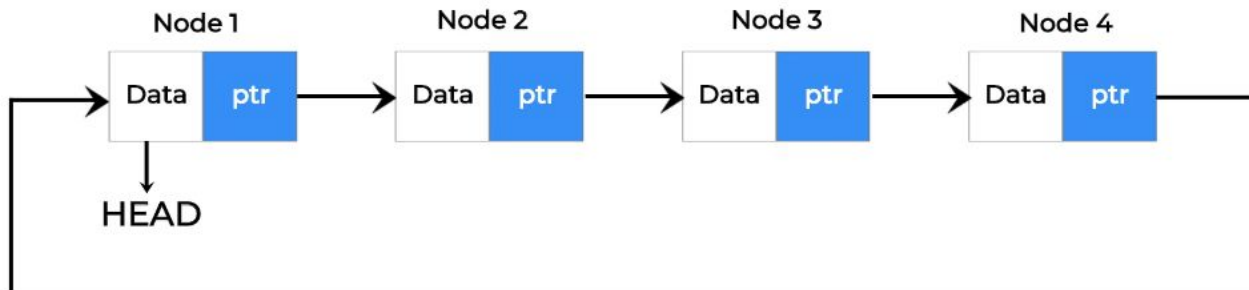
A node can be added in three ways:

1. insert a new node in an empty list
2. Insertion at the beginning of the list
3. Insertion at the end of the list
4. Insertion in between the nodes

```
#include<iostream>
using namespace std;

typedef struct Node
{
    int data;
    Node *next;
}Node;
```

Circular Linked List in C++



// Create a new node

```
node* createNode(int data)
{
    node* newNode = new node;
    if (newNode == NULL)
    {
        cout<<"Failed to allocate memory for a new node";
        exit(1);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```


// Insert a node at the beginning of the circular linked list

```
node* insertAtBeginning(node* head, int data)
{
    node* newNode = new node;
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL)
    {
        newNode->next = newNode; // Points to itself
        head = newNode;
    }
    else
    {
        node* current = head;
        while (current->next != head) {
            current = current->next;
        }
        newNode->next = head;
        current->next = newNode;
        head = newNode;
    }
}
```

// Insert a node at the end of the circular Linked List

```
node* insertAtEnd(node* head, int data) {
    node* newNode = new node;
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->next = newNode; // Points to itself
        head = newNode;
    }
    else
    {
        node* current = head;
        while (current->next != head) {
            current = current->next;
        }
        newNode->next = head;
        current->next = newNode;
    }
    return head;
}
```

// Function to insert a new node at the specified position of a DLL

```
node *InsertAtPos(node *head, int value, int position)
```

```
{
```

```
    if(position < 1)
```

```
    {
```

```
        cout<<"Invalid Position";
```

```
        return head;
```

```
    }
```

```
    if(position == 1)
```

```
        return insertAtBeginning(head, value);
```

```
    node *ptr1 = new node;
```

```
    ptr1 = head;
```

```
    node *ptr2 = new node;
```

```
    ptr2->data = value;
```

```
    ptr2->next = NULL;
```

```
    while(--position > 1)
```

```
        ptr1 = ptr1->next;
```

```
    ptr2->next = ptr1->next;
```

```
    ptr1->next = ptr2;
```

```
    return head;
```

// Delete a node with a given data value from the circular linked list

```
node* deleteNode(node* head, int data)
{
    if (head == NULL)
    {
        printf("List is empty, nothing to delete.\n");
        return head;
    }

    node* current = head;
    node* prev = NULL;

    do {
        if (current->data == data)
        {
            if (current == head && current->next == head)
            {
                // Only one node in the circular list
                head = NULL;
            }
        }
    } while (current->next != head);

    if (prev != NULL)
        prev->next = current->next;
    else
        head = current->next;

    free(current);
    return head;
}
```

```
else if (current == head)
{
    // Removing the head node
    node* t = head;
    while (t->next != head)
    {
        t = t->next;
    }
    head = current->next;
    t->next = head;
}
else
{
    prev->next = current->next;
}
delete current;
return head;
}
prev = current;
current = current->next;
} while (current != head);

printf("Element not found in the list.\n");
return head;
```

```
// Display the circular linked list
```

```
node* displayList(node* head)
```

```
{
```

```
    node* current = head;
```

```
    if (head == NULL)
```

```
    {
```

```
        printf("Circular Linked List is empty.\n");
```

```
        return head;
```

```
    }
```

```
    printf("Circular Linked List: ");
```

```
    do
```

```
    {
```

```
        printf("%d -> ", current->data);
```

```
        current = current->next;
```

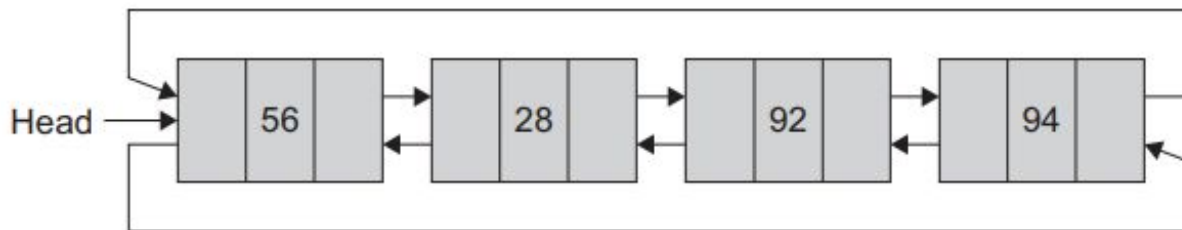
```
    } while (current != head);
```

```
    printf(" (head)\n");
```

```
}
```

Doubly Circular Linked List

- In doubly circular linked list, the last node's next link is set to the first node of the list and the first node's previous link is set to the last node of the list. This gives access to the last node directly from the first node



The operations on circular linked list—insert, delete, create and traverse—follow the same method as that of linear list except for a few changes.

POLYNOMIAL MANIPULATIONS

Representation of single variable polynomials using linked list -

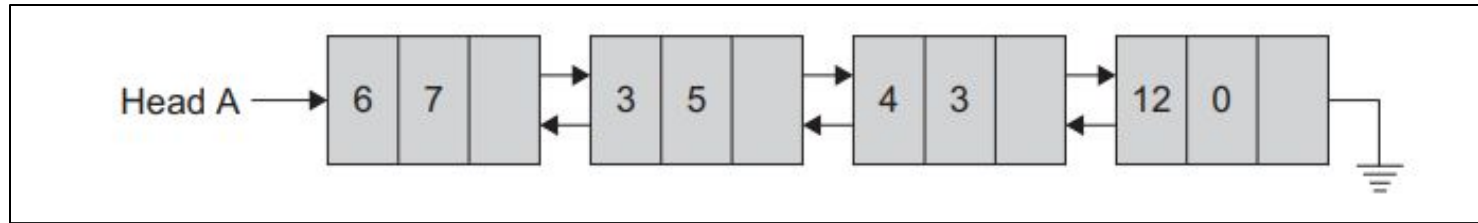
- The manipulation of symbolic polynomials is a good application of list processing.
- Let the polynomial we want to represent using a linked list be $A(x)$. It is expanded as,

$$A(x) = k_1x^m + \dots + k_{n-1}x^2 + k_nx^1$$

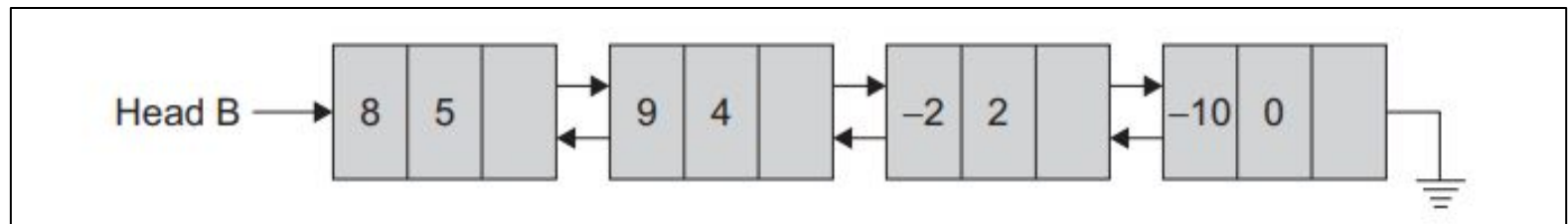
- where k_i is a non-zero coefficient with exponent m such that $m > m-1 > \dots > 2 > 1 \geq 0$.
- A node of the linked list will represent each term.
- A node will have 3 fields, which represent the coefficient and exponent of a term and a pointer to the next term

Coefficient	Exponent	Link
-------------	----------	------

For instance, the polynomial, say $A = 6x^7 + 3x^5 + 4x^3 + 12$ would be stored as



The polynomial $B = 8x^5 + 9x^4 - 2x^2 - 10$ would be stored as



```
#include<iostream>
using namespace std;
typedef struct Node {
    int coef;
    int exp;
    struct Node* next;
}Node;

void insert(Node** poly, int coef, int exp) {
    Node* temp = new Node;
    temp->coef = coef;
    temp->exp = exp;
    temp->next = NULL;

    if (*poly == NULL) {
        *poly = temp;
        return;
    }
    Node* current = *poly;
    while (current->next != NULL) {
        current = current->next;
    }
}
```

```
void print(Node* poly)
{
    if (poly == NULL)
    {
        cout<<"0\n";
        return;
    }

    Node* current = poly;

    while (current != NULL)
    {
        cout<<current->coef<<"x^"<<current->exp;
        if (current->next != NULL)
        {
            cout<<" + ";
        }
        current = current->next;
    }

    cout<<"\n";
}
```

```
Node* add(Node* poly1, Node* poly2) {
    Node* result = NULL;

    while (poly1 != NULL && poly2 != NULL) {
        if (poly1->exp == poly2->exp) {
            insert(&result, poly1->coef + poly2->coef, poly1->exp);
            poly1 = poly1->next;
            poly2 = poly2->next;
        } else if (poly1->exp > poly2->exp) {
            insert(&result, poly1->coef, poly1->exp);
            poly1 = poly1->next;
        } else {
            insert(&result, poly2->coef, poly2->exp);
            poly2 = poly2->next;
        }
    }
}
```

```
while (poly1 != NULL)
{
    insert(&result, poly1->coef, poly1->exp);
    poly1 = poly1->next;
}

while (poly2 != NULL)
{
    insert(&result, poly2->coef, poly2->exp);
    poly2 = poly2->next;
}

return result;
}
```

```
int main() {  
    Node* poly1 = NULL;  
    insert(&poly1, 5, 4);  
    insert(&poly1, 3, 2);  
    insert(&poly1, 1, 0);  
  
    Node* poly2 = NULL;  
    insert(&poly2, 4, 4);  
    insert(&poly2, 2, 2);  
    insert(&poly2, 1, 1);  
  
    cout<<"First polynomial: ";  
    print(poly1);  
  
    cout<<"Second polynomial: ";  
    print(poly2);  
  
    Node* result = add(poly1, poly2);  
    cout<<"Result: ";  
    print(result);  
  
    return 0;  
}
```

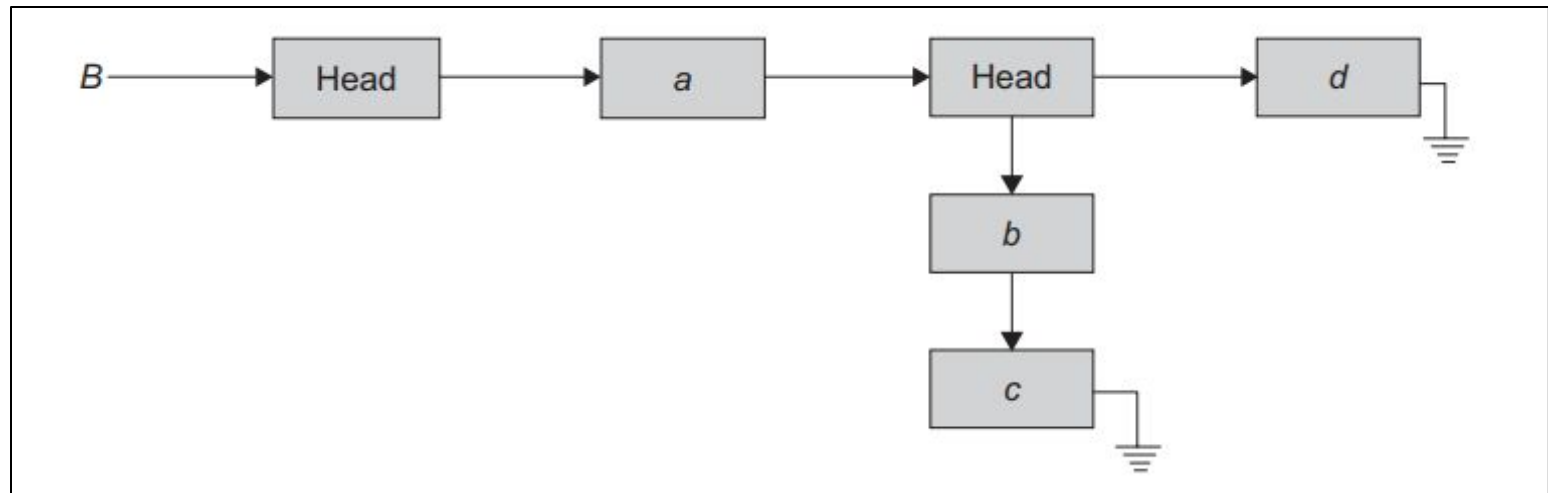
Generalized Linked List

- Generalized lists are defined recursively as lists whose members may be single data elements or other generalized lists.
- Generalized lists are the most flexible and useful structures
- A generalized list is a linear list (non-indexed) of zero or more data elements or generalized lists.
- In other words, a generalized list is a finite sequence of $n \geq 0$ elements, $a_1, a_2, \dots, a_n$, which we write as list $A = (a_1, a_2, \dots, a_n)$, where a_i is either an atom or the list.
- The elements of a_i , where $1 \leq i \leq n$, which are not atoms are said to be the sub-lists of the list.
- Here A is the name of generalized list and n is its length.

Some examples of generalized lists are the following:

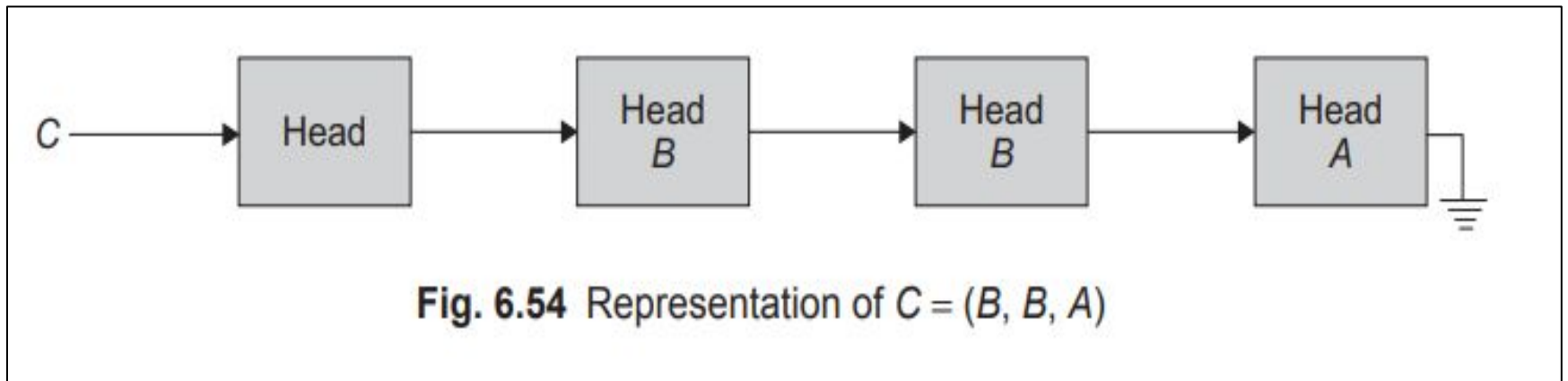
1. $A = ()$ - The empty (or null) list.
2. $B = (a, (b, c), d)$ - List of three elements—the first element is a, the second element is list (b, c), and the third element is d.
3. $C = (B, B, A)$ - List of length 3 with the first and the second element as list B and the third element as list A, which is a null list.
4. $D = (a, b, D)$ - List of length 3 which is recursive as it includes itself as one of the elements. It can also be written as
 $D = (a, b, (a, b, (a, b, \dots))\dots$

- In example 2, A is a list made up of three components. The first component is an atom, the second component is a list made up of two atoms, and the third component is the atom d.
- One of the better approaches to visualize the generalized lists is using a header node. In this approach, each generalized list has a header node labelled Head. Figure shows the pictorial representation of list B.



Representation of B = (a, (b, c), d)

- In example 3, the list C has three components: the first component is list B, the second component is again list B, and the third component is list A.
- This can be pictorially viewed as in Fig.



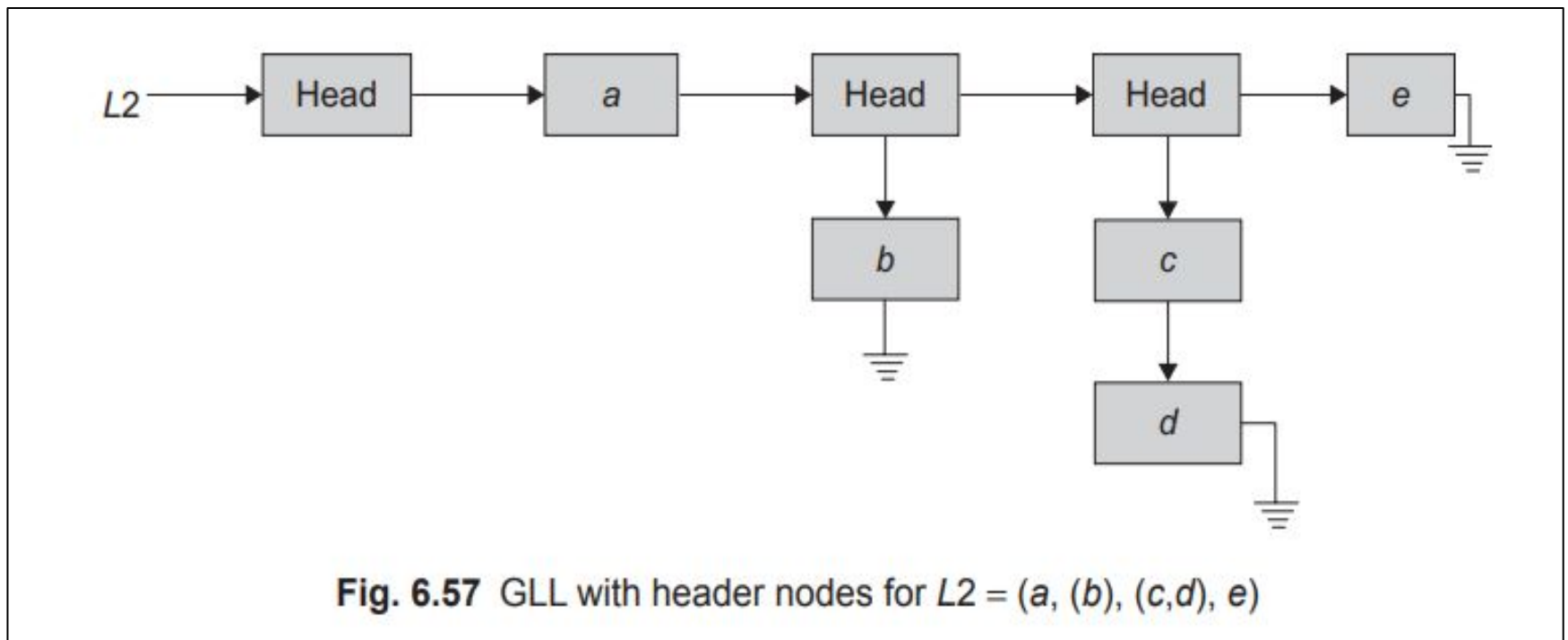
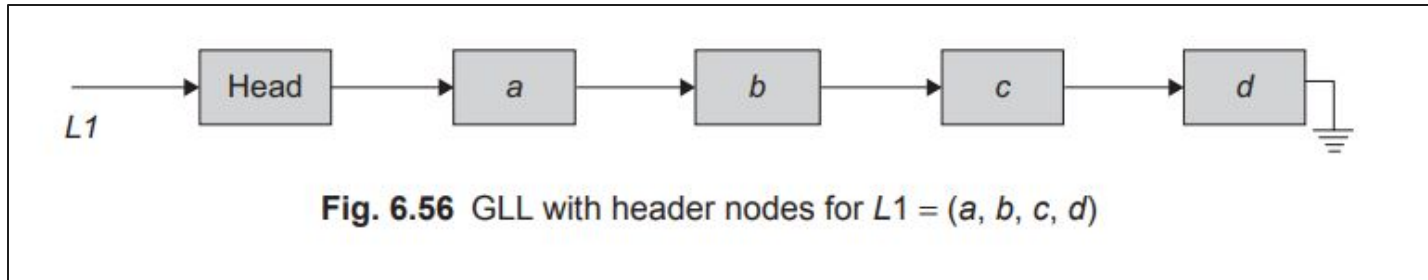
- The list D in example 4 can be viewed as Fig.



Fig. 6.55 Representation of $D = (a, b, D)$

These cases represent the categories of generalized lists in the order of implementation complexity. The lists could be one of the following categories:

- **Lists with no shared references**—The components of one list are not members of any other list. In example 2, B is a list with no shared references.
- **Lists with shared references**—The components of one list can be the members of another list. The logical interpretation of lists leads to two categories as the following:
 1. **Static interpretation**—The current status of the referenced list is anticipated. The referenced list is copied into the referencing list.
 2. **Dynamic interpretation**—The list itself is anticipated. Any future changes in the referenced list should be reflected in the referencing list.
- **Recursive list**—A recursive list is the one that directly or indirectly references itself. Here D is a recursive list.



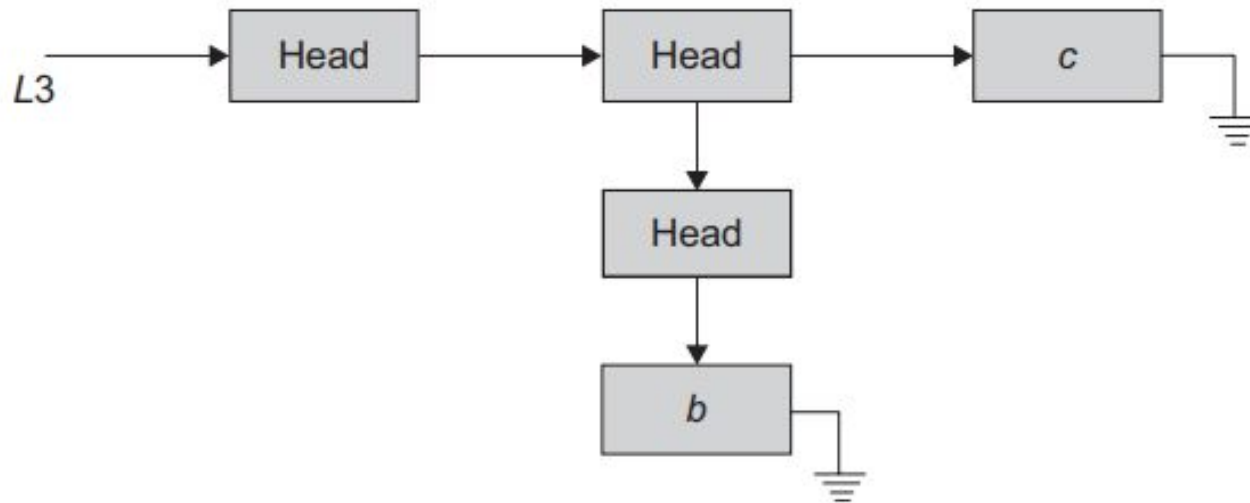
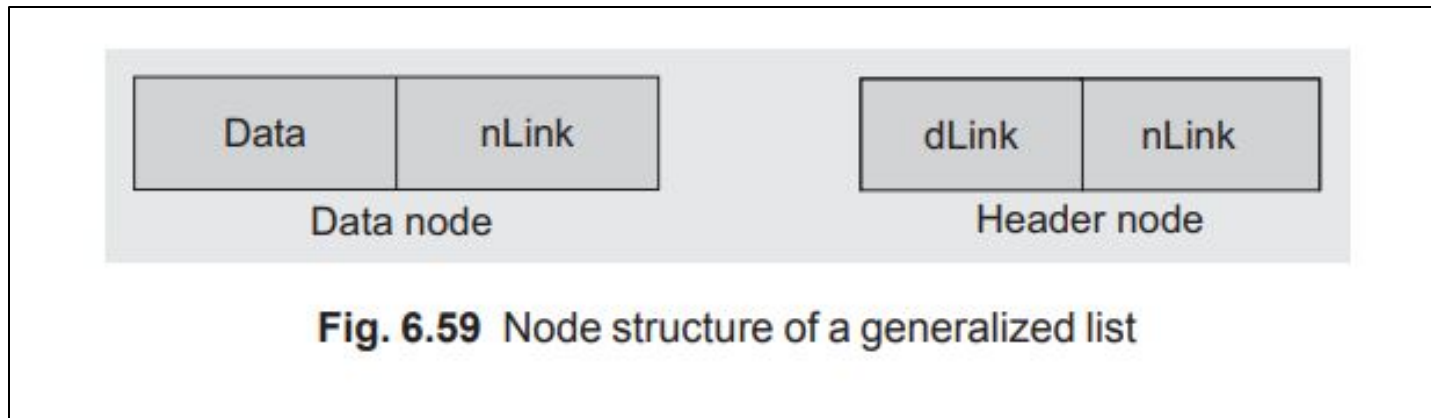
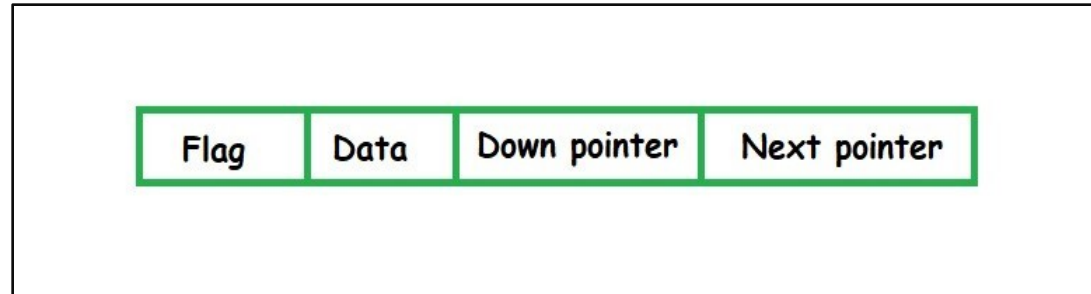


Fig. 6.58 GLL with header nodes for $L3 = (a, ((b)), c)$

- In a linked representation of the generalized list, each node has fields as either
 1. **Data and nLink**—the data field(s) would store data and the nLink field refers to the next member node which could be an atom or a list or
 2. **Header node** that has two links dlink and nLink, where dlink is used to refer to the first node of the list member (which could be an atom or a list) and the nLink refers to the next member node (which could be an atom or a list)



- A Generalized Linked List L , is defined as a finite sequence of $n \geq 0$ elements, $l_1, l_2, l_3, l_4, \dots, l_n$, such that l_i are either item or the list of items. Thus $L = (l_1, l_2, l_3, l_4, \dots, l_n)$ where n is total number of nodes in the list.



To represent a list of items there are certain assumptions about the node structure.

1. **Flag = 1** implies that down pointer exists
2. **Flag = 0** implies that next pointer exists
3. **Data** means the item.
4. **Down pointer** is the address of node which is down of the current node
5. **Next pointer** is the address of node which is attached as the next node

Representation of Polynomials Using Generalized linked list

- We often need to process a polynomial with more than one variable.
- Consider the following polynomial P with three variables x ,y, and z.
- Consider the two-variable polynomial Q of x and y.

$$Q(x, y) = 5x^4y^3 + 6x^6y^5 + 3x^5y^2 + xy$$

- Now, similar to a single variable polynomial, we can represent this polynomial Q(x,y) as a sequential organization with four fields: coefficient, Exp_X, Exp_Y, and nLink as



Fig. 6.61 Two-variable polynomial

Similarly, for $P(x, y, z) = 9x^8y^2z + 4x^4y^3z^3 + x^6y^5z^4 + 8x^5y^2z + 7x^4y^6z + 4xyz + 3xz$ we can represent this polynomial $P(x, y, z)$ as a sequential organization with five fields: coefficient, Exp_X, Exp_Y, Exp_Z, and nLink as in Fig. 6.62.

Coefficient	Exp_X	Exp_Y	Exp_Z	nLink
-------------	-------	-------	-------	-------

Fig. 6.62 Three-variable polynomial